FORTIFY

Insurance Management Platform

Software Requirements Specification

Section 1 — Introduction

1.1 Purpose

This Software Requirements Specification defines the complete functional and non-functional requirements for Fortify, a modern full-stack Insurance Management Platform. The document serves as the primary technical and business reference for the development team, system architects, quality assurance engineers, and project stakeholders involved in the design, development, and validation of the system. It establishes what the system must do, the constraints under which it must operate, the data it must manage, and the security guarantees it must uphold.

1.2 Scope

Fortify is a web-based commercial insurance management system that digitizes the entire insurance lifecycle from user onboarding through policy issuance to claim settlement. The platform serves four distinct user categories — Admins, Policyholders, Agents, and Claim Officers — each operating within a role-specific portal with dedicated workflows.

The system covers the following primary functional domains:

- User registration through public request submission and admin-controlled approval with automated credential dispatch
- Insurance product catalogue management including the creation and governance of insurance types and policy definitions
- Policy application submission, multi-stage review workflow, and agent-driven approval with automated contract activation
- Dynamic premium calculation using business risk profiling based on industry classification, revenue, and workforce size
- Claims lifecycle management from initial filing through investigation, evidence collection, and final settlement
- Secure authentication using JWT tokens with role-based access control and OTP-driven password recovery
- Payment tracking for premium collections, claim settlements, and policy refunds

The following are explicitly outside the scope of the current specification: real-time payment gateway integration, automated fraud scoring through machine learning, WebSocket-based push notifications, and cloud-based document storage. These are identified as planned future enhancements.

1.3 Definitions, Acronyms, and Abbreviations

SRS — Software Requirements Specification — this document

JWT — JSON Web Token — a compact URL-safe token used for stateless authentication per RFC 7519

RBAC — Role-Based Access Control — a security model that grants system permissions based on assigned user roles

OTP — One-Time Password — a time-limited numeric code sent via email for password reset verification

API — Application Programming Interface — the contract through which the frontend communicates with the backend

REST — Representational State Transfer — the architectural style governing API design

JPA — Java Persistence API — the specification used for object-relational mapping with the relational database

ORM — Object-Relational Mapping — the technique of mapping Java entity classes to relational database tables

DTO — Data Transfer Object — a plain Java class used to decouple API payload contracts from internal entity models

BCrypt — A password hashing function used to securely encode user credentials before database storage

CORS — Cross-Origin Resource Sharing — a browser security mechanism governing cross-origin HTTP request permissions

GL — General Liability — a category of commercial insurance covering third-party bodily injury and property damage

Policyholder — A commercial business entity that purchases insurance coverage through the Fortify platform

Agent — A licensed insurance professional assigned to review and process policy applications

Claim Officer — A staff member responsible for investigating and settling insurance claims

Admin — The system operator responsible for user governance and platform configuration

Premium — The periodic payment made by a policyholder to maintain active insurance coverage

Coverage Limit — The maximum amount the insurer will pay on an approved claim

Deductible — The amount the policyholder must bear personally before insurance coverage activates

1.4 References

- Angular 18 Official Documentation — https://angular.dev
- Spring Boot 3.x Reference Guide — https://spring.io/projects/spring-boot
- JSON Web Token Specification RFC 7519 — https://www.rfc-editor.org/rfc/rfc7519
- OWASP Top Ten Web Application Security Risks — https://owasp.org/www-project-top-ten
- Tailwind CSS Documentation — https://tailwindcss.com/docs
- Spring Security Reference Documentation — https://docs.spring.io/spring-security

1.5 Document Overview

Section 2 provides the overall system description including operating context, user classes, and general constraints. Section 3 details functional requirements organized by user role. Section 4 covers non-functional requirements across performance, usability, reliability, and scalability. Section 5 describes the system architecture and design constraints. Section 6 specifies external interface requirements for the UI, API, and email service. Section 7 defines the data requirements and full entity model. Section 8 addresses security requirements comprehensively. Section 9 contains appendices with status flow diagrams, identifier formats, and API reference material.

Section 2 — Overall System Description

2.1 Product Perspective

Fortify is a standalone web application built on a decoupled frontend and backend architecture. The Angular 18 frontend operates as a single-page application communicating exclusively with the Spring Boot backend through a RESTful JSON API. The system does not depend on any external insurance platform, third-party policy engine, or legacy system. All data persistence is managed internally through a relational database accessed via JPA and Hibernate.

During development the Angular CLI server runs on port 4200 and communicates with the Spring Boot API on port 8080 through a configured proxy. In production both layers are served from a unified deployment environment. All authenticated communication is secured using JWT tokens transmitted in HTTP Authorization headers.

2.2 Product Functions

At the highest level of abstraction Fortify performs the following core functions:

1. Governs user access through a structured onboarding workflow beginning with a public registration request and culminating in admin-controlled credential issuance
2. Maintains a searchable product catalogue of insurance policies grouped by insurance type and filterable by multiple simultaneous criteria
3. Orchestrates a multi-stage policy application workflow passing through policyholder submission, admin assignment ,agent review and approval
4. Calculates insurance premiums dynamically at the point of application using a risk-based algorithm incorporating the policyholder's business profile
5. Manages the full claims lifecycle from incident filing through documentary evidence collection, officer investigation, and final settlement or rejection
6. Enforces strict role-based access control ensuring each user interacts exclusively with data and operations relevant to their designated function
7. Tracks all monetary flows on the platform including premium payments, claim settlements, and policy refunds
8. Provides secure credential recovery through an OTP-based password reset mechanism with enforced complexity requirements

2.3 User Classes and Characteristics

2.3.1 Admin

The Admin is the system operator and governance authority. Admins are provisioned directly in the system during initial deployment and are not subject to the public registration workflow. They have unrestricted access to all management functions including user approval, policy catalogue administration, workflow orchestration, and platform audit oversight. Admins do not participate in insurance transactions themselves but govern every workflow that enables other roles to operate.

2.3.2 Policyholder

The Policyholder represents a commercial business entity seeking insurance coverage. Policyholders are onboarded through the public registration form and require admin approval before gaining system access. Once active they interact with the platform to discover and apply for policies, manage their business profile, monitor application status, file claims against active policies, and track payment history. Policyholders are the primary revenue-generating users and their experience quality directly reflects the platform's commercial value.

2.3.3 Agent

The Agent is a licensed insurance professional assigned to review and process policy applications. Agents are onboarded through the registration request workflow and may carry a defined specialization used by the admin to match them with appropriate policyholders. Their primary responsibility is evaluating the policyholder's business profile and application details then exercising professional judgment to forward the application to an agent. Agents also monitor their portfolio of active and pending cases.

2.3.4 Claim Officer

The Claim Officer is an internal staff member responsible for the investigation and settlement of insurance claims. Claim officers are assigned to specific claims by the admin based on regional coverage or subject matter expertise. They review incident descriptions, examine uploaded evidence documents, and determine the appropriate settlement amount or rejection basis. Their decisions carry direct financial consequences and must be supported by formal justification recorded in the system.

2.4 Operating Environment

The frontend is an Angular 18 application delivered to modern web browsers supporting ES2020 or later including current versions of Chrome, Firefox, Safari, and Edge. The backend requires a Java 17 or later runtime with Spring Boot 3.x. The database layer targets any JPA-compatible relational database with MySQL 8.x or PostgreSQL 14 or later recommended for production. An SMTP-capable email service must be available for credential dispatch and OTP delivery. The development environment uses the Angular CLI development server and Spring Boot's embedded Tomcat with a proxy configuration bridging ports 4200 and 8080.

2.5 Assumptions and Dependencies

- An SMTP-capable email service is configured and available for all system-generated emails
- All users access the system through a supported modern web browser with JavaScript enabled
- The admin user account is pre-seeded in the database at deployment time with a known and secure initial credential
- Business profile information provided by policyholders during registration is assumed accurate for premium calculation purposes
- The Angular frontend correctly attaches the JWT token to all authenticated API requests via a configured HTTP interceptor
- Network connectivity between the application server and the email service is available and reliable

2.6 General Constraints

- Passwords must never be stored in plaintext and must be encoded using BCrypt at a minimum work factor of ten
- Policy start and end dates must be null until the moment of agent approval
- Premium calculations must be performed server-side and must never be accepted from client-supplied values
- User records with associated policies, applications, or claims must not be hard-deleted from the system
- All API responses must use standardized error structures and must not expose internal stack traces or database messages

Section 3 — Functional Requirements

3.1 Admin — System Governance

3.1.1 Registration Request Management

The admin must be able to view all registration requests in PENDING status with full applicant details. On approval the system must atomically perform the following: transition the user status to ACTIVE, generate a temporary password in the format Shield@XXXX where XXXX is a random four-digit number, encode that password using BCrypt, record the approving admin identity and timestamp, create the role-specific profile record, and dispatch a welcome email containing login credentials. For a POLICYHOLDER a BusinessProfile record is created from the registration details. For an AGENT an Agent profile is created with auto-generated agent code and default commission rate. For a CLAIM_OFFICER a ClaimOfficer profile is created with an auto-generated employee code. On rejection the system sets the status to REJECTED, records the admin remarks, and dispatches a rejection notification email.

3.1.2 User Lifecycle Management

The admin must be able to view all users filterable by role and status, deactivate active users by setting status to INACTIVE which immediately prevents authentication, and reactivate inactive accounts. User records must never be permanently deleted when they carry associated policies, applications, or claims.

3.1.3 Insurance Type Management

The admin must be able to create insurance type categories with a name, description, and active flag. The admin may toggle the active status of any insurance type. When deactivated all policies belonging to that type must be excluded from policyholder browse and search results.

3.1.4 Policy Product Management

The admin creates insurance policies under a specified insurance type. The creation form accepts the policy name, coverage description, minimum and maximum coverage amounts, fixed base premium, and duration in months. The system auto-generates the policy number using a prefix derived from the first four characters of the insurance type name in uppercase followed by a zero-padded four-digit sequence number. The admin must not manually supply the policy number.

New policies default to active on creation. The admin may toggle any policy between active and inactive. Policies with existing applications must not be hard-deleted.

3.1.5 Policy Application Assignment

The admin views all SUBMITTED applications and assigns an agent to each. The system records the assignment, calculates the agent commission as a percentage of the premium using the agent's configured commission rate, and advances the application status to ASSIGNED.

3.1.6 Claim Officer Assignment

The admin views all SUBMITTED claims and assigns a claim officer. The system records the assignment, advances the claim status to ASSIGNED, and records the assignment timestamp.

3.2 Policyholder — Client Portal

3.2.1 Policy Discovery and Search

An authenticated policyholder may browse all active policies or apply advanced filters through a POST search endpoint. Accepted filter parameters include a list of insurance type names, a premium range with minimum and maximum bounds, a coverage range with minimum and maximum bounds, a list of duration values in months, a list of keywords matched case-insensitively against both policy name and description, and a sort directive specifying field and direction. Any combination of these parameters may be applied simultaneously. When no parameters are supplied all active policies from active insurance types are returned.

3.2.2 Policy Application Submission

A policyholder submits an application by specifying the target policy and a desired coverage amount within the policy's allowed range. The system automatically retrieves the policyholder's business profile using the authenticated identity. The profile must exist and its completion flag must be true. The system rejects duplicate applications where a prior application for the same policy exists in any non-REJECTED status. On successful validation the system calculates the server-side premium, stores the application with status SUBMITTED, null agent, null start and end dates, and an auto-generated application number.

3.2.3 Premium Calculation Algorithm

The base premium defined on the selected policy is multiplied by a risk multiplier beginning at 1.0. Employee count contributes an increment of 0.1 for more than 20 employees, 0.2 for more than 50, and 0.3 for more than 100, with only the highest applicable increment applied. Annual revenue contributes 0.05 for more than one million, 0.15 for more than five million, and 0.25 for more than ten million, again applying only the highest applicable increment. Industry classification contributes 0.4 for construction or manufacturing, 0.2 for retail or restaurant, and 0.1 for technology or consulting. The final premium is rounded to two decimal places using half-up rounding.

3.2.4 Application Status Tracking

A policyholder must be able to view all their submitted applications showing application number, policy name, insurance type, current status, calculated premium, and where approved, the start and end dates. The status progression visible to the policyholder is SUBMITTED, ASSIGNED, UNDER_REVIEW, then either ACTIVE on approval or REJECTED with a displayed rejection reason.

3.2.5 Claim Filing

A policyholder may file a claim against any ACTIVE policy application. The submission requires an incident description, the incident date and location, and an estimated loss amount. The system validates that the incident date falls within the policy's coverage window and that the claim amount does not exceed the coverage limit. On success the system stores the claim with an auto-generated claim number, status SUBMITTED, null officer assignment, null approved amount, and the current filing timestamp.

3.2.6 Claim Tracking and Document Upload

A policyholder must be able to view all their filed claims with current status, incident details, and where settled, the approved payout. Rejected claims must display the rejection reason. Policyholders must be able to upload supporting documents to any of their claims at any point before settlement.

3.2.7 Business Profile Management

A policyholder must be able to view and update their business profile including business name, industry, annual revenue, employee count, and city. Profile updates do not retroactively affect premium amounts on existing applications but apply to all new applications submitted after the update.

3.3 Agent — Risk Assessment

3.3.1 Application Queue and Review

An agent must be able to view all policy applications assigned to them filterable by status. For each application the agent may view the full details including the policyholder's complete business profile, the selected policy terms, coverage amount, and calculated premium. Based on this review the agent submits the application, which records the assignment and advances the status to UNDER_REVIEW. Agents may only act on ASSIGNED applications.

3.3.2 Portfolio Monitoring

An agent's dashboard must display the count of applications in each status, the total premium value of active policies under their management, and recent activity across their assigned portfolio.

3.4 Claim Officer — Settlement

3.4.1 Assigned Claims and Investigation

A claim officer must be able to view all claims assigned to them organized by status. The officer may advance a claim to UNDER_INVESTIGATION status to signal active review. During investigation the officer may access all uploaded claim documents, review the policy details and coverage limits, and add investigation notes.

3.4.2 Settlement Decision

After investigation the claim officer makes a final determination. A full approval specifies the approved payout amount which must not exceed the coverage limit. A partial approval specifies a reduced amount with a written justification. A rejection provides a documented reason. On any

approval decision the system creates a SETTLEMENT payment record linked to the claim. On full settlement the claim status advances to SETTLED and the settlement timestamp is recorded.

3.5 Authentication and Password Management

3.5.1 Login

All users authenticate through a single shared login endpoint by providing their registered email address and password. The system verifies the account exists, that the status is ACTIVE, and that the password matches the BCrypt hash. A PENDING user receives a message indicating that their account awaits admin approval. An INACTIVE or REJECTED user receives an appropriate denial message. On success the system returns a JWT token, the user role, full name, email, and internal identifier.

3.5.2 OTP Password Reset

A user initiates password reset by submitting their registered email. The system generates a numeric OTP valid for ten minutes and dispatches it by email. The user submits the OTP and a new password meeting the complexity requirement of at least one uppercase letter, one lowercase letter, one digit, and one special character. On successful OTP verification the system encodes and stores the new password and immediately invalidates the OTP to prevent reuse.

Section 4 — Non-Functional Requirements

4.1 Performance

Standard read operations including policy browsing, application status queries, and claim list retrieval must return responses within 500 milliseconds under normal load. Complex filtered search queries involving multiple simultaneous parameters must complete within 1000 milliseconds. The premium calculation algorithm must execute synchronously within the application submission request without introducing perceptible latency. The system must support at least 100 concurrent authenticated users without degrading below these thresholds. All list endpoints with potentially large result sets must support pagination.

4.2 Usability

The Angular frontend must provide immediate visual feedback on all user interactions including form submissions, status transitions, file uploads, and search operations. The Signals-based reactivity model must ensure UI updates are isolated to only the affected components. Status badges must use a consistent color coding scheme applied uniformly across all dashboards. The platform must support both dark and light themes with the preference persisted across sessions. Role-specific dashboards must present actionable summary metrics immediately on login without requiring additional navigation.

4.3 Reliability and Availability

The system must handle unexpected or malformed input gracefully without crashing the application server. All multi-step write operations, particularly the registration approval workflow that creates both a user record and a role profile, must execute within database transactions to prevent partial failure states. Auto-generated identifiers including policy numbers, application numbers, and claim numbers must be unique system-wide and their generation must be safe under concurrent submission scenarios. The application server must be stateless so that the system can be horizontally scaled without session affinity requirements.

4.4 Maintainability

The backend must follow a strict layered architecture separating controller, service, repository, entity, and DTO concerns. Business logic including premium calculation, status transition rules, and commission computation must reside exclusively in the service layer. Entities must not be serialized directly to API responses. The premium calculation must be an isolated private method modifiable without affecting surrounding workflow code. The Angular frontend must organize each role's features into a dedicated module or folder so that role-specific changes do not affect unrelated parts of the application. Shared components must be extracted into a shared module and reused across role features.

4.5 Scalability

The database schema must include indexes on all foreign key columns and high-frequency query fields including user status, policy application status, and claim status. The filtered policy search must use the JPA Specification pattern to construct queries at the database level rather than filtering in application memory. Query performance must not degrade below specified response time thresholds as data volumes grow.

4.6 Compatibility

The frontend must function correctly on current stable releases of Chrome, Firefox, Safari, and Edge. The API must respond with standard HTTP status codes and JSON payloads compatible with any REST client. The backend must be deployable on any environment supporting Java 17 or later with a compatible relational database.

Section 5 — System Architecture and Design Constraints

5.1 Architectural Overview

Fortify follows a three-tier architecture comprising a presentation layer implemented as an Angular 18 single-page application, an application layer implemented as a Spring Boot RESTful API, and a data layer consisting of a relational database accessed through JPA and Hibernate. The presentation and application layers are physically decoupled and communicate exclusively through HTTP JSON messages. No server-side rendering is used; the backend serves only data, never markup.

5.2 Frontend Architecture

The Angular frontend uses standalone components as the primary building block, eliminating the need for NgModule declarations for feature components. State management is implemented using the Angular Signals API ensuring that reactive state updates propagate only to the components consuming the affected signals. HTTP communication is handled through Angular's HttpClient with an interceptor that automatically attaches the JWT Authorization header to all outgoing requests. Reactive Forms are used for all data entry screens enabling real-time validation feedback. The design system uses Tailwind CSS with glassmorphism card surfaces, micro-animations on state transitions, and togglable dark and light color themes.

5.3 Backend Architecture

The Spring Boot backend is organized into mandatory layers. The controller layer contains REST controllers with role-based access annotations mapping HTTP requests to service calls. The service layer contains all business logic, workflow orchestration, premium calculation, and status transition enforcement. The repository layer contains JPA repository interfaces with custom query methods and specification-based dynamic filtering. The entity layer contains JPA-annotated domain classes mapping to database tables. The DTO layer contains plain Java classes serving as all API input and output contracts.

Spring Security intercepts all requests, validates JWT tokens, and enforces role-based method security using the PreAuthorize annotation. The JWT utility class handles token generation,

parsing, and validation. BCrypt password encoding is configured as a Spring Bean and injected into services handling credential operations.

5.4 Mandatory Design Constraints

- The service layer is the only layer that may contain business logic; controllers must remain thin and entities must be free of business rules
- Entity objects must never be returned directly from controller methods; all API responses must use DTOs
- The JPA Specification pattern must be used for all dynamic multi-criteria queries
- All multi-table write operations must be annotated with the Spring Transactional annotation
- Role-based access control must be enforced at the controller method level using PreAuthorize and must not rely solely on frontend route guards
- The frontend must never trust or display premium amounts calculated or modified on the client side
- The Angular dev proxy must be the only mechanism bridging frontend and backend ports in development and must not be applied in production builds

Section 6 — External Interface Requirements

6.1 User Interface

The user interface is an Angular 18 single-page application delivered to the browser. The application presents a public landing page, a registration request form, and a shared login page without role selection. After authentication the application reads the role from the JWT response and routes the user automatically to the appropriate role-specific dashboard. The design language features glassmorphism card surfaces, smooth micro-animations on interactions, and a togglable dark and light color scheme implemented through Tailwind CSS utility classes.

6.2 API Interface

The backend exposes a RESTful HTTP API accepting and returning JSON. HTTP GET retrieves data. HTTP POST creates resources and handles complex filtered search operations. HTTP PUT and PATCH update resources and trigger status transitions. The API returns standard HTTP status codes: 200 for success, 201 for resource creation, 400 for validation errors, 401 for authentication failures, 403 for authorization failures, 404 for resource not found, and 500 for server errors. All error responses follow a uniform structure containing an error code and a human-readable message. Endpoints except the public registration and login endpoints require a valid JWT Bearer token in the Authorization header.

6.3 Email Interface

The system dispatches emails through an SMTP service on three trigger events: admin approval of a registration request sends a welcome email with login credentials, admin rejection of a registration request sends a notification with the rejection reason, and initiation of a password reset sends an OTP. All email dispatch must be handled asynchronously so that SMTP latency or temporary unavailability does not block synchronous API operations or return errors to the calling client.

6.4 Development Proxy Interface

During development the Angular CLI proxy configuration forwards all requests from port 4200 matching the /api path prefix to the Spring Boot backend on port 8080. This eliminates cross-

origin issues during local development. The proxy configuration must be maintained in proxy.conf.json and must not be applied in production builds where CORS configuration or a reverse proxy handles routing.

Section 7 — Data Requirements and Entity Model

7.1 Entity Overview

The Fortify data model consists of twelve primary entities organized into three logical groups. Identity and profile entities represent the people using the system. Product and transaction entities represent the insurance products and their purchase lifecycle. Financial and evidence entities support the claims and payment processes.

7.2 Identity and Profile Entities

7.2.1 Users

The central identity record for every person on the platform regardless of role. Stores full name, unique email address, BCrypt-encoded password, phone number, assigned role, and account status. Also stores business fields — business name, industry, annual revenue, employee count, and city — captured during registration for prospective policyholders. Additional fields include registration message, admin remarks, a reference to the reviewing admin, and review timestamp. The created-at timestamp is set automatically by a JPA pre-persist lifecycle hook. The status field follows the progression PENDING on registration, then ACTIVE on approval or REJECTED on rejection, and may transition to INACTIVE by admin action.

7.2.2 Business Profiles

Created automatically when an admin approves a policyholder registration. Holds a one-to-one reference to the User record and stores business name, industry classification, annual revenue, employee count, city, and a boolean profile completion flag. This entity is the primary input to the premium calculation algorithm.

7.2.3 Agent

Stores the professional profile of an insurance agent with a one-to-one reference to the User record, a system-generated agent code in the format AGT-XXXX, a license number, territorial coverage, specialization classification, and commission rate expressed as a percentage.

7.2.4 Claim Officer

Stores the professional profile of a claim officer with a one-to-one reference to the User record, a system-generated employee code in the format CO-XXXX, and a regional coverage designation.

7.3 Product and Transaction Entities

7.3.1 Insurance Types

Defines the top-level categories of insurance products available on the platform. Each record holds a type name, description, and active flag. Examples include Property Insurance, General Liability, Cyber Insurance, Auto Insurance, and Workers Compensation. Insurance types serve as the primary browsing and filtering dimension for policyholders.

7.3.2 Policies

Defines specific insurance products available for purchase. Each policy belongs to one InsuranceType and carries a system-generated policy number, policy name, coverage description, exclusions text, minimum and maximum coverage amounts, fixed base premium, standard duration in months, and active flag. Policies contain no policyholder-specific data and no date fields. They represent the product definition that policyholders browse and agents and reference during application review.

7.3.3 Policy Application

The central transaction record of the insurance lifecycle. Created when a policyholder applies for a specific policy. Holds references to the policyholder User, selected Policy, policyholder Business Profile, assigned Agent. Stores the selected coverage amount, server-calculated premium, agent commission amount, policyholder notes, and rejection reason. The status field progresses through SUBMITTED, ASSIGNED, UNDER_REVIEW, then either ACTIVE or REJECTED. Start and end dates are null until agent approval and are set exclusively at that moment using the policy's duration.

7.4 Financial and Evidence Entities

7.4.1 Claims

Represents an insurance claim filed against an active PolicyApplication. Holds references to the PolicyApplication, filing policyholder, and assigned ClaimOfficer. Stores incident date, location, description, estimated loss amount, approved payout amount, and rejection reason. The status field progresses through SUBMITTED, ASSIGNED, UNDER_INVESTIGATION, then APPROVED, PARTIALLY_APPROVED, REJECTED, or SETTLED. Timestamps track filing, assignment, and settlement.

7.4.2 Claim Documents

Stores references to files uploaded as evidence for a claim. Each record holds a reference to the parent Claim, the identity of the uploader, document type classification, file name, storage path, file type, file size in bytes, and upload timestamp. Document type accepts values PHOTO, VIDEO, INVOICE, POLICE_REPORT, MEDICAL_REPORT, and OTHER.

7.4.3 Payments

Records all monetary transactions on the platform. Each record references the associated Policy Application and captures the transaction amount, payment method, transaction reference number, payment date, payment type, and status. The payment type field distinguishes PREMIUM payments flowing from policyholder to insurer, SETTLEMENT payments flowing from insurer to policyholder following an approved claim, and REFUND payments returned to the policyholder on policy cancellation. Payment status accepts PENDING, SUCCESS, FAILED, and REFUNDED.

7.4.4 Policy Documents

Stores formal policy contract documents associated with an approved Policy Application. Each record references the parent Policy Application and stores file name, storage path, file type, and file size.

7.5 Entity Relationships

Users has a one-to-one relationship with each of Business Profiles, Agent, and Claim Officer. A single Insurance Type has a one-to-many relationship with Policies. A Policy has a one-to-many relationship with Policy Applications. A Policy Application belongs to one Policy, one User as policyholder, one Business Profile, one Agent, and has one-to-many relationships with Claims, Payments, and Policy Documents. A Claim belongs to one Policy Application and one Claim Officer and has a one-to-many relationship with Claim Documents.

Section 8 — Security Requirements

8.1 Authentication

All API endpoints except the public registration and login endpoints must require a valid JWT Bearer token in the Authorization request header. JWT validation must verify the token's digital signature, confirm the token has not expired, and extract the user email and role claims. A token failing any check must result in a 401 Unauthorized response. Tokens must carry a configurable expiry period and must not be issued indefinitely. The system must be stateless with no server-side session state; all authentication context must be carried in the token.

8.2 Authorization

Role-based access control must be enforced at the controller method level using Spring Security's PreAuthorize annotation. A policyholder must not invoke any admin, agent, or claim officer endpoint. An agent must not invoke claim officer endpoints. A claim officer must not invoke agent endpoints. These constraints must be enforced server-side and must not rely on frontend routing alone. Unauthorized role access must return a 403 Forbidden response.

Data-level access control must also be enforced in service logic. A policyholder must only retrieve their own applications and claims. An agent must only view applications assigned to them. A claim officer must only act on claims assigned to them. These restrictions must be enforced through scoped repository queries and must not be bypassable through identifier manipulation in the request.

8.3 Password Security

All passwords must be encoded using BCrypt at a minimum work factor of ten before storage. Plaintext passwords must never appear in database records, application logs, API responses, or error messages. Temporary onboarding passwords must be transmitted only through email and must be changed by the user at first login. Password reset OTPs must be valid for a maximum of ten minutes and must be invalidated immediately upon successful use and must not be reusable.

New passwords must comply with the following complexity requirements: minimum length of eight characters, at least one uppercase letter, at least one lowercase letter, at least one numeric digit,

and at least one special character. Non-compliant passwords must be rejected with a descriptive validation message identifying the violated rule.

8.4 Data Protection

The password field must be excluded from all DTO serialization. The convertToDTO methods must never copy the password value from the entity to the DTO. API responses must not include internal implementation details, Hibernate error messages, stack traces, or any information that could assist an attacker in understanding the system's internals.

CORS must be configured on the Spring Boot backend to accept requests only from the registered frontend origin. Requests from unregistered origins must be rejected by the CORS filter before reaching application logic. In production the allowed origin must be set to the production frontend domain and must never be configured as a wildcard.

8.5 Input Validation

All external input must be validated at the service layer before persistence. Validation must cover the presence of required fields, email address format, numeric bounds of coverage amounts against policy limits, temporal validity of incident dates against policy coverage periods, and role-appropriateness of submitted data. Validation failures must return 400 Bad Request responses with descriptive messages identifying the invalid field and the violated rule.

8.6 Audit Trail

The system maintains an implicit audit trail through the data model. All approval and rejection decisions must record the identity of the deciding admin and the decision timestamp. All status transitions on policy applications and claims must be traceable through status fields and associated timestamp fields. Created-at timestamps on Users, PolicyApplications, and Claims must be set automatically by JPA lifecycle hooks and must not be modifiable through the API.

Section 9 — Appendices

Appendix A — Status Flow Reference

A.1 User Account Status Flow

A user account begins in PENDING status when the registration request is submitted. Admin approval transitions the status to ACTIVE permitting authentication. Admin rejection transitions the status to REJECTED. An admin may set any ACTIVE account to INACTIVE at any time immediately suspending authentication. An INACTIVE account may be restored to ACTIVE.

- PENDING → ACTIVE (admin approval)
- PENDING → REJECTED (admin rejection)
- ACTIVE → INACTIVE (admin deactivation)
- INACTIVE → ACTIVE (admin reactivation)

A.2 Policy Application Status Flow

SUBMITTED is the initial status set on application submission. ASSIGNED is set when the admin assigns an agent. UNDER_REVIEW is set when the agent approves the application. ACTIVE is set on agent approval simultaneously populating start and end dates. REJECTED is set on agent rejection with the reason recorded. EXPIRED is a future automated transition when the current date passes the end date. CANCELLED may be set by the admin.

- SUBMITTED → ASSIGNED → UNDER_REVIEW → ACTIVE → EXPIRED
- SUBMITTED → ASSIGNED → UNDER_REVIEW → REJECTED
- ACTIVE → CANCELLED

A.3 Claim Status Flow

SUBMITTED is the initial status set on claim filing. ASSIGNED is set when the admin assigns a claim officer. UNDER_INVESTIGATION is set when the officer begins active review. APPROVED is set on full approval. PARTIALLY_APPROVED is set on partial approval with justification. REJECTED is set on denial with documented reason. SETTLED is the terminal status set when settlement payment is confirmed.

- SUBMITTED → ASSIGNED → UNDER_INVESTIGATION → APPROVED → SETTLED

- SUBMITTED → ASSIGNED → UNDER_INVESTIGATION → PARTIALLY_APPROVED → SETTLED
- SUBMITTED → ASSIGNED → UNDER_INVESTIGATION → REJECTED

Appendix B — Auto-Generated Identifier Formats

Policy Number — POL-{TYPE}-{NNNN} — Example: POL-PROP-0001, POL-LIAB-0002, POL-CYBE-0001

Application Number — APP-{NNNN} — Example: APP-0023

Claim Number — CLM-{NNNN} — Example: CLM-0007

Agent Code — AGT-{NNNN} — Example: AGT-0004

Claim Officer Employee Code — CO-{NNNN} — Example: CO-0003

Temporary Password — Shield@{DDDD} — Example: Shield@4521

Appendix C — Premium Risk Multiplier Reference

The base risk multiplier starts at 1.0. The following increments are applied additively. Within each category only the highest applicable threshold increment is applied.

Employee Count: Greater than 20 adds 0.1. Greater than 50 adds 0.2. Greater than 100 adds 0.3.

Annual Revenue: Greater than 1,000,000 adds 0.05. Greater than 5,000,000 adds 0.15. Greater than 10,000,000 adds 0.25.

Industry Classification: Construction or Manufacturing adds 0.4. Retail or Restaurant adds 0.2. Technology or Consulting adds 0.1. Unmatched industries add 0.0.

Appendix D — Payment Type Reference

PREMIUM — Money flowing from the policyholder to the insurance company in satisfaction of a periodic premium obligation

SETTLEMENT — Money flowing from the insurance company to the policyholder following an approved or partially approved claim

REFUND — Money returned from the insurance company to the policyholder upon policy cancellation before the end of the coverage period

Appendix E — API Endpoint Summary

Authentication: POST /api/auth/login, POST /api/auth/request-registration, POST /api/auth/forgot-password, POST /api/auth/reset-password

User Management: GET /api/users, GET /api/users/{id}, GET /api/users/pending, PATCH /api/users/{id}/approve, PATCH /api/users/{id}/reject, PATCH /api/users/{id}/deactivate, PATCH /api/users/{id}/activate

Insurance Types: POST /api/insurance-types, GET /api/insurance-types, GET /api/insurance-types/active, GET /api/insurance-types/{id}, PATCH /api/insurance-types/{id}/toggle-status

Policy Catalogue: POST /api/policies, GET /api/policies, GET /api/policies/{id}, PATCH /api/policies/{id}/toggle-status

Policy Discovery: POST /api/policyholder/policies/search, GET /api/policyholder/policies/{id}

Policy Applications: POST /api/policyholder/apply, GET /api/policyholder/my-applications, PATCH /api/policy-applications/{id}/assign-agent

Claims: POST /api/policyholder/claims/raise, GET /api/policyholder/claims, PATCH /api/claims/{id}/assign-officer, PATCH /api/claims/{id}/investigate, PATCH /api/claims/{id}/approve, PATCH /api/claims/{id}/partially-approve, PATCH /api/claims/{id}/reject, PATCH /api/claims/{id}/settle

Payments: POST /api/payments, GET /api/payments/{id}, GET /api/payments/policy-application/{id}

Documents: POST /api/claim-documents, GET /api/claim-documents/claim/{id}, POST /api/policy-documents, GET /api/policy-documents/policy-application/{id}

Business Profiles: GET /api/business-profiles/user/{userId}, PUT /api/business-profiles/{id}

Appendix F — Status Badge Color Reference

SUBMITTED — Blue — initial filing state

ASSIGNED — Yellow — resource has been assigned, awaiting action

UNDER_REVIEW or UNDER_INVESTIGATION — Orange — active review in progress

APPROVED or ACTIVE — Green — approved or live coverage

PARTIALLY_APPROVED — Teal — approved with reduced payout

REJECTED — Red — denied with reason recorded

SETTLED — Purple — claim fully resolved and paid

EXPIRED or CANCELLED or INACTIVE — Grey — terminal or suspended state